Module 4: Evaluation

Function Scoping

Conjunction junction, what's your function?

1.

Function Resolution

Switching to your secondary scope is faster than reloading.

Function Resolution

Definition

Resolution is the process of looking up a function (or variable) in scope.

- Variables use just their name.
- Functions use their **signature**, which includes more than name.
- Note: Functions may or may not have the same scope as variables.
  - JavaScript: Same Scope
  - Java: Different Scopes

Signature

The function **signature** is the information used to uniquely identify/resolve a function.

- Shared scope with variables:
  - **Name** (or **symbol**, e.g. **+**): JavaScript, Lisp
  - Typically results from first-class functions.
- Unique scope for functions:
  - **Arity** (# of parameters): Elixir
  - **Parameter Types**: Java
  - **Return Type**: Haskell (via inference)
  - Corresponds to **overloading**.

Static vs Dynamic Dispatch

Static Dispatch

In **static** dispatch, the invoked function is resolved with *compile-time* types.

- This is *functions* in most langs.
  - Relevant when parameter types have **"disjoint"** overloads.
  - `visit(Ast)/visit(Ast.Let)`
- Easy to implement; effectively just "go to wherever the impl code is".

Dynamic Dispatch

In **dynamic** dispatch, the invoked function is resolved with *runtime* types.

- This is the foundation of *methods* in OOP.
  - The definition of a method is determined based on the object - aka polymorphism.
  - `ast.toString()/let.toString()`
- Hard(er) to implement; requires storing and looking up method definitions at runtime.
- "Double dispatch" is often used to refer to dynamic dispatch for arguments as well.
  - This is the point of the Visitor pattern!

```java
//with static dispatch:

void visit(Ast ast) { … }
void visit(Ast.Let ast) { … }

Ast ast = new Ast.Let(…);
Ast.Let let = new Ast.Let(…);

visit(ast); //visit Ast
visit(let); //visit Let
visit((Ast) let); //visit Ast

ast.toString(); //Ast.toString() !!!
let.toString(); //Let.toString()
((Ast) let).toString(); //Ast !!!
```

```java
//with dynamic dispatch:

void visit(Ast ast) { … }
void visit(Ast.Let ast) { … }

Ast ast = new Ast.Let(…);
Ast.Let let = new Ast.Let(…);

visit(ast); //visit Let !!!
visit(let); //visit Let
visit((Ast) let); //visit Let !!!

ast.toString(); //Let.toString()
let.toString(); //Let.toString()
((Ast) let).toString(); //Let
```

Static vs Dynamic Dispatch

2.

Scoping & Closures

“We have functional programming at home.”

Static vs Dynamic Scoping

Static Scoping

In **static** scoping, scope hierarchy is determined by *program structure*.

- The parent is the scope where the function is defined.
- Provides a consistent, compile-time relation between use/declaration.
  - Necessary for semantic analysis!
- Used by nearly all languages.

Dynamic Scoping

In **dynamic** scoping, scope hierarchy is determined by *runtime context* (call stack).

- The parent is the scope where the function is called at *runtime*.
  - Cannot be determined at compile-time.
  - Can break caller's variable encapsulation!
- Hard/impossible for humans to follow.
- Used by Perl (ish) and broken Evaluators.
  - React's `useContext()` and dependency injection are effectively dynamic scoping.

```
//with static scoping:

LET scope = "definition";

DEF function() DO
  //child of definition
  print("child of " + scope);
END

DEF main() DO
  LET scope = "invocation";
  function();
END

main(); //child of definition
```

```
//with dynamic scoping:

LET scope = "definition";

DEF function() DO
  //child of ???
  print("child of " + scope);
END

DEF main() DO
  LET scope = "invocation";
  function();
END

main(); //child of invocation
```

Function Scopes

Where is dynamic scoping actually used?

~~Perl (Raku)~~, Bash, LaTeX

Dependency Injection, e.g. useContext

Lambdas

Lambda

A **lambda** is an anonymous function used to store/pass executable code.

- The body of a lambda can then be executed later, just like functions.
  - Most beneficial for langs without nested/first-class functions.
- Often has subtle differences or restrictions compared to functions.
  - e.g. JavaScript “this”.

Capture vs Closure

Functions access outer variables either by **capturing** values or **“closing over”** scopes.

- Capture: The lambda stores/uses the variable value at the time of definition.
- **Closure**: The lambda resolves the current variable value each time it is accessed.
  - Effectively, it “closes over” the definition scope and has access to parent variables.
- Affects behavior when mutating variables.
  - Java bypasses this with “effectively final”.

```
//with capture:

LET variable = "original";

DEF function() DO
  //capture variable = "original"
  print("value is " + variable);
END

variable = "final";

function(); //value is original
```

```
//with closure:

LET variable = "original";

DEF function() DO
  //"close over" variable scope
  print("value is " + variable);
END

variable = "final";

function(); //value is final
```

Capture vs Closure

```go
var funcs []func()
for i := 1; i ⩽ 3; i++ {
  funcs = append(funcs, func() {
    fmt.Println(i)
  })
}
for _, f := range funcs { f() }

//capture: 1, 2, 3
//closure: 4, 4, 4 !!!

//Go 1.21:
{ i := <int>, { body… }
              { body… }
              { body… } }
```

```go
var funcs []func()
for i := 1; i ⩽ 3; i++ {
  i := i //capture i in new scope
  funcs = append(funcs, func() {
    fmt.Println(i)
  })
}
for _, f := range funcs { f() }

//closure: 1, 2, 3

//Go 1.22:
{ i := 1, { body… } }
{ i := 2, { body… } }
{ i := 3, { body… } }
```

Fixing For Loops In Go

Module 4: Evaluation

Function Scoping

Conjunction junction, what's your function?

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals